

03__cross__condition

June 24, 2026

1 Step 3: Directed Zero-Shot Cross-Condition Transfer (Authoritative v4)

This notebook keeps the fuller repository-facing cross-condition analysis flow for the authoritative v4 line. The central question is whether a classifier trained to separate one disease from healthy controls can transfer **zero-shot** to a different disease-versus-control task when subject-level evaluation remains primary.

```
[1]: import json
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from IPython.display import display
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix

def find_repo_root() -> Path:
    candidates = [Path.cwd().resolve(), *Path.cwd().resolve().parents]
    for base in candidates:
        if (base / 'data' / 'processed' / 'v4' / 'v4_protocol_manifest.json').
            exists():
            return base
    raise RuntimeError('Could not locate repository root from the current_
working directory.')

REPO_ROOT = find_repo_root()
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))
if str(REPO_ROOT / 'src') not in sys.path:
    sys.path.insert(0, str(REPO_ROOT / 'src'))
STEP_TAG = 'step3'
FIG_PDF = REPO_ROOT / 'report' / 'figures' / 'v4' / 'pdf' / STEP_TAG
```

```

FIG_PNG = REPO_ROOT / 'report' / 'figures' / 'v4' / 'png' / STEP_TAG
TABLE_DIR = REPO_ROOT / 'report' / 'tables' / 'v4'
for directory in (FIG_PDF, FIG_PNG, TABLE_DIR):
    directory.mkdir(parents=True, exist_ok=True)

plt.rcParams.update({
    'font.family': 'DejaVu Serif',
    'font.size': 9,
    'axes.titlesize': 9,
    'axes.labelsize': 9,
    'xtick.labelsize': 8,
    'ytick.labelsize': 8,
    'legend.fontsize': 7,
    'figure.dpi': 180,
    'savefig.bbox': 'tight',
    'pdf.fonttype': 42,
    'ps.fonttype': 42,
})

IEEE_ONE_COL = 3.5
IEEE_TWO_COL = 7.16
DIRECTIONS = ['pd_to_hd', 'hd_to_pd', 'pd_to_als', 'als_to_pd', 'hd_to_als',
               ↪ 'als_to_hd']
DIR_LABEL = {d: d.replace('_to_', '→').upper() for d in DIRECTIONS}
COND_ORDER = ['pd', 'hd', 'als']
COND_LABEL = {'pd': 'PD', 'hd': 'HD', 'als': 'ALS'}
COND_COLOR = {'pd': '#4C78A8', 'hd': '#F58518', 'als': '#54A24B'}
CLF_ORDER = ['rf', 'knn', 'svm', 'dt', 'qda', 'xgb', 'lgbm']
CLF_LABEL = {'rf': 'RF', 'knn': 'KNN', 'svm': 'SVM', 'dt': 'DT', 'qda': 'QDA',
              ↪ 'xgb': 'XGBoost', 'lgbm': 'LightGBM'}

def savefig_stem(fig: plt.Figure, stem: str) -> None:
    pdf_path = FIG_PDF / f'{stem}.pdf'
    png_path = FIG_PNG / f'{stem}.png'
    fig.savefig(pdf_path)
    fig.savefig(png_path, dpi=300)
    print(f'Saved figure: {pdf_path.relative_to(REPO_ROOT)}')
    print(f'Saved figure: {png_path.relative_to(REPO_ROOT)}')

def save_table_csv(df: pd.DataFrame, name: str, *, index: bool = False) -> None:
    path = TABLE_DIR / name
    df.to_csv(path, index=index)
    print(f'Saved table: {path.relative_to(REPO_ROOT)}')

```

```

BASE = REPO_ROOT / 'experiments' / 'results' / 'v4'
cross_results = json.loads((BASE / 'cross_condition_results_v4.json').
    ↪read_text())
within_results = {condition: json.loads((BASE / f'{condition}_results_v4.json').
    ↪read_text()) for condition in COND_ORDER}

rows = []
for direction in DIRECTIONS:
    source, target = direction.split('_to_')
    payload = cross_results[direction]
    for clf_name, entry in payload['classifiers'].items():
        rows.append({
            'direction': direction,
            'source': source,
            'target': target,
            'classifier': clf_name,
            'subject_f1': float(entry['subject_primary_f1_macro']),
            'stride_f1': float(entry['f1_macro']),
            'within_subject_f1': float(entry['within_subject_f1_macro']),
            'within_stride_f1': float(entry['within_stride_f1_macro']),
            'delta_subject': float(entry['delta_f1_subject']),
            'delta_stride': float(entry['delta_f1_stride']),
            'subject_ci_low': float(entry['subject_primary_f1_ci_lower']),
            'subject_ci_high': float(entry['subject_primary_f1_ci_upper']),
            'stride_ci_low': float(entry['f1_macro_ci_lower']),
            'stride_ci_high': float(entry['f1_macro_ci_upper']),
            'control_specificity_subject': ␣
            ↪float(entry['control_b_specificity_subject']),
            'target_recall_subject': ␣
            ↪float(entry['target_disease_recall_subject']),
            'control_specificity_stride': ␣
            ↪float(entry['control_b_specificity_stride']),
            'target_recall_stride': ␣
            ↪float(entry['target_disease_recall_stride']),
            'selected_strategy': entry.get('selected_imbalance_strategy', entry.
            ↪get('selected_resampling')),
            'permutation_p_value': float(entry.get('permutation_p_value', np.
            ↪nan)),
        })
transfer_df = pd.DataFrame(rows)
save_table_csv(transfer_df, 'step3_cross_condition_summary_v4.csv')

best_by_direction = (
    transfer_df
    .sort_values(['direction', 'subject_f1', 'stride_f1'], ascending=[True, ␣
    ↪False, False])

```

```

        .groupby('direction')
        .head(1)
        .copy()
    )
    best_by_direction['direction_label'] = best_by_direction['direction'].
        ↪map(DIR_LABEL)

    print('Best classifier per direction under the subject-primary endpoint:')
    for row in best_by_direction.itertuples():
        print(f"    {row.direction_label}: {CLF_LABEL[row.classifier]} (subject_
        ↪F1={row.subject_f1:.4f}, matched degradation={row.delta_subject:+.4f})")

```

Saved table: report/tables/v4/step3_cross_condition_summary_v4.csv

Best classifier per direction under the subject-primary endpoint:

```

ALS→HD: LightGBM (subject F1=0.8052, matched degradation=+0.0392)
ALS→PD: DT (subject F1=0.8600, matched degradation=-0.0156)
HD→ALS: QDA (subject F1=0.8929, matched degradation=-0.0216)
HD→PD: QDA (subject F1=0.8083, matched degradation=+0.0629)
PD→ALS: QDA (subject F1=0.9481, matched degradation=-0.0881)
PD→HD: QDA (subject F1=0.8712, matched degradation=-0.0112)

```

1.1 Cross-condition degradation table

This table preserves the full **42-case** transfer surface with one row per (direction, classifier). The most important result is direction-level rather than cell-level: the **mean subject-level matched degradation is positive in all six directions**. The largest direction-average losses occur in **HD→ALS (0.1925)** and **HD→PD (0.1735)**, while **PD→HD** is the least degraded on average (**0.0458**). In this notebook, matched degradation = within-source subject F1 - cross-condition transfer subject F1, so larger positive values mean more loss under transfer. The key caveat is that some individual classifier-direction rows are still near zero or negative, so the paper should not claim that every pair degrades.

```

[2]: table_df = transfer_df.copy()
    table_df['Direction'] = table_df['direction'].map(DIR_LABEL)
    table_df['Classifier'] = table_df['classifier'].map(CLF_LABEL)
    display_cols = [
        'Direction', 'Classifier', 'subject_f1', 'within_subject_f1',
        ↪'delta_subject',
        'stride_f1', 'within_stride_f1', 'delta_stride', 'subject_ci_low',
        ↪'subject_ci_high',
        'control_specificity_subject', 'target_recall_subject',
        ↪'selected_strategy', 'permutation_p_value'
    ]
    styled = (
        table_df[display_cols]
        .style
        .format({
            'subject_f1': '{:.4f}',

```

```

        'within_subject_f1': '{:.4f}',
        'delta_subject': '{:+.4f}',
        'stride_f1': '{:.4f}',
        'within_stride_f1': '{:.4f}',
        'delta_stride': '{:+.4f}',
        'subject_ci_low': '{:.4f}',
        'subject_ci_high': '{:.4f}',
        'control_specificity_subject': '{:.4f}',
        'target_recall_subject': '{:.4f}',
        'permutation_p_value': '{:.4f}',
    })
    .background_gradient(subset=['delta_subject'], cmap='RdYlBu_r', vmin=-0.15,
↪vmax=0.40)
    .background_gradient(subset=['subject_f1'], cmap='Blues', vmin=0.45, vmax=1.
↪0)
    .set_caption('Cross-condition transfer summary (subject-primary columns,
↪emphasized)')
)
display(styled)

```

<pandas.io.formats.style.Styler at 0x7f378bc992e0>

1.2 Per-subject stride accuracy under the best direction-level classifier

This panel is diagnostic rather than headline-making. It shows that even when a direction looks strong in aggregate, the retained per-subject stride accuracy can still be heterogeneous across target-disease subjects and controls. That heterogeneity matters because the v4 claim is not that transfer works uniformly for every held-out individual, only that zero-shot transfer is **measurable** and sometimes practically strong at the subject level.

```

[3]: subject_rows = []
    for row in best_by_direction.itertuples():
        entry = cross_results[row.direction]['classifiers'][row.classifier]
        y_true = np.asarray(entry['y_true'], dtype=int)
        y_pred = np.asarray(entry['y_pred'], dtype=int)
        subject_ids = np.asarray(cross_results[row.direction]['target_subject_ids'])
        for subject_id in np.unique(subject_ids):
            mask = subject_ids == subject_id
            yt = y_true[mask]
            yp = y_pred[mask]
            subject_rows.append({
                'direction': row.direction,
                'direction_label': DIR_LABEL[row.direction],
                'classifier': row.classifier,
                'subject_id': subject_id,
                'label': int(yt[0]),
                'accuracy': float(np.mean(yt == yp)),
                'n_strides': int(mask.sum()),
            })

```

```

    })
subject_df = pd.DataFrame(subject_rows)
save_table_csv(subject_df, 'step3_subject_accuracy_v4.csv')

fig, axes = plt.subplots(2, 3, figsize=(IEEE_TWO_COL, 5.6), sharex=False,
    ↪sharey=True)
axes = axes.ravel()
for ax, direction in zip(axes, DIRECTIONS):
    frame = subject_df[subject_df['direction'] == direction].
    ↪sort_values(['label', 'accuracy', 'subject_id'], ascending=[False, True,
    ↪True]).reset_index(drop=True)
    colors = ['#C44E52' if label == 1 else '#4C78A8' for label in
    ↪frame['label']]
    ax.bar(np.arange(len(frame)), frame['accuracy'], color=colors,
    ↪edgecolor='black', linewidth=0.35)
    ax.set_title(f"{DIR_LABEL[direction]} / {CLF_LABEL[best_by_direction.
    ↪set_index('direction').loc[direction, 'classifier']]}")
    ax.set_xticks(range(len(frame)))
    ax.set_xticklabels(frame['subject_id'], rotation=75, ha='right', fontsize=6)
    ax.set_ylim(0, 1.02)
    ax.grid(axis='y', alpha=0.2)
handles = [
    plt.Line2D([0], [0], color='#C44E52', lw=6, label='Target disease subject'),
    plt.Line2D([0], [0], color='#4C78A8', lw=6, label='Control B subject'),
]
fig.legend(handles=handles, loc='upper center', bbox_to_anchor=(0.5, 1.03),
    ↪ncol=2, frameon=False)
fig.suptitle('Per-subject stride accuracy under the best subject-primary
    ↪classifier for each direction', y=1.08)
fig.tight_layout()
savefig_stem(fig, 'step3_per_subject_accuracy')
plt.show()

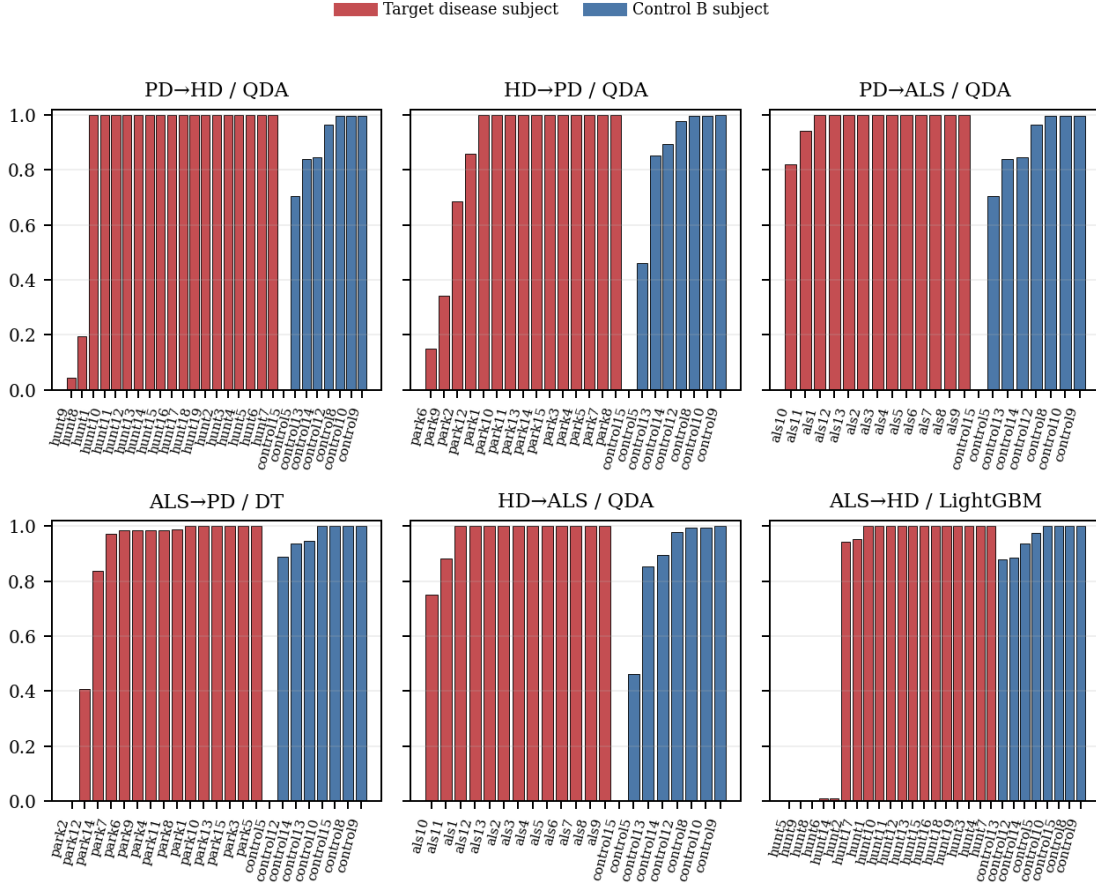
```

Saved table: report/tables/v4/step3_subject_accuracy_v4.csv

Saved figure: report/figures/v4/pdf/step3/step3_per_subject_accuracy.pdf

Saved figure: report/figures/v4/png/step3/step3_per_subject_accuracy.png

Per-subject stride accuracy under the best subject-primary classifier for each direction



1.3 Heatmaps: matched degradation, cross-condition subject F1, and best-direction transfer matrix

These heatmaps answer different questions and therefore deserve to coexist. **degradation_heatmap** is the main comparison figure because it shows how much each classifier-direction pairing loses relative to its own within-source benchmark. **cross_condition_f1_heatmap** complements that view with absolute subject-level transfer performance. **step3_best_direction_heatmap** is a compact direction-only summary. A concrete v4 pattern worth keeping explicit is that **QDA** leads four of the six directions, while **DT** leads **ALS→PD** and **LightGBM** leads **ALS→HD**. The single strongest absolute subject-level transfer result is **QDA** on **PD→ALS** at **0.9481**, followed by **QDA** on **HD→ALS** at **0.8929** and **QDA** on **PD→HD** at **0.8712**.

```
[4]: delta_subject_mat = np.array([
    [cross_results[direction]['classifiers'][clf]['delta_f1_subject'] for
    direction in DIRECTIONS]
    for clf in CLF_ORDER
])
```

```

subject_f1_mat = np.array([
    [cross_results[direction]['classifiers'][clf]['subject_primary_f1_macro']]
    ↪for direction in DIRECTIONS
    ↪for clf in CLF_ORDER
])

best_mat = np.full((3, 3), np.nan)
for row in best_by_direction.itertuples():
    i = COND_ORDER.index(row.source)
    j = COND_ORDER.index(row.target)
    best_mat[i, j] = row.subject_f1

fig, ax = plt.subplots(figsize=(IEEE_TWO_COL * 0.92, 3.5),
    ↪constrained_layout=True)
sns.heatmap(
    pd.DataFrame(delta_subject_mat, index=[CLF_LABEL[c] for c in CLF_ORDER],
    ↪columns=[DIR_LABEL[d] for d in DIRECTIONS]),
    cmap='RdBu_r',
    center=0.0,
    annot=True,
    fmt='+.3f',
    linewidths=0.5,
    linecolor='white',
    cbar_kws={'label': 'Matched subject-level degradation (within - cross)'},
    ax=ax,
)
ax.set_title('Matched subject-level degradation by classifier and direction')
ax.set_xlabel('Transfer direction')
ax.set_ylabel('Classifier')
savefig_stem(fig, 'degradation_heatmap')
plt.show()

fig, ax = plt.subplots(figsize=(IEEE_TWO_COL * 0.92, 3.5),
    ↪constrained_layout=True)
sns.heatmap(
    pd.DataFrame(subject_f1_mat, index=[CLF_LABEL[c] for c in CLF_ORDER],
    ↪columns=[DIR_LABEL[d] for d in DIRECTIONS]),
    cmap=sns.light_palette('#0B3C5D', as_cmap=True),
    vmin=0.4,
    vmax=1.0,
    annot=True,
    fmt='.3f',
    linewidths=0.5,
    linecolor='white',
    cbar_kws={'label': 'Cross-condition subject F1'},
    ax=ax,
)

```



```

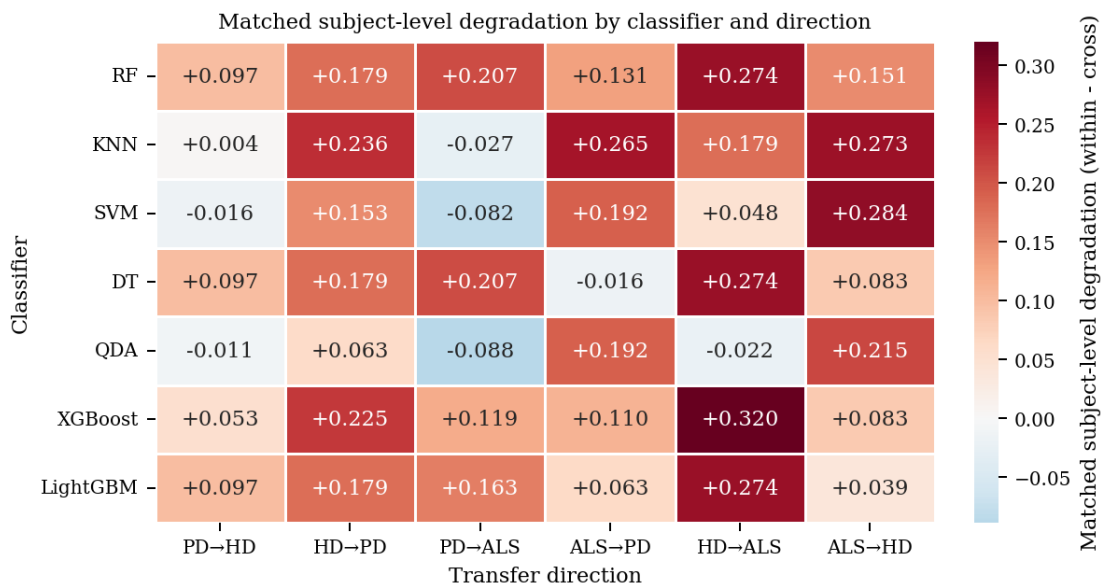
ax.set_title('Cross-condition subject-level macro-F1 by classifier and_
↳direction')
ax.set_xlabel('Transfer direction')
ax.set_ylabel('Classifier')
savefig_stem(fig, 'cross_condition_f1_heatmap')
plt.show()

best_df = pd.DataFrame(best_mat, index=[COND_LABEL[c] for c in COND_ORDER],_
↳columns=[COND_LABEL[c] for c in COND_ORDER])
fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.1, 3.0),_
↳constrained_layout=True)
sns.heatmap(
    best_df,
    cmap=sns.light_palette('#0B3C5D', as_cmap=True),
    vmin=0.4,
    vmax=1.0,
    annot=False,
    linewidths=0.5,
    linecolor='white',
    cbar_kws={'label': 'Best-direction subject F1'},
    ax=ax,
)
for i, source in enumerate(COND_ORDER):
    for j, target in enumerate(COND_ORDER):
        if i == j or np.isnan(best_mat[i, j]):
            continue
        row = best_by_direction[(best_by_direction['source'] == source) &_
↳(best_by_direction['target'] == target)].iloc[0]
        ax.text(
            j + 0.5,
            i + 0.5,
            f"{best_mat[i, j]:.3f}\n{CLF_LABEL[row['classifier']]}",
            ha='center',
            va='center',
            fontsize=7,
            color='white' if best_mat[i, j] > 0.72 else 'black',
        )
ax.set_title('Best classifier and subject F1 for each transfer direction')
ax.set_xlabel('Target condition')
ax.set_ylabel('Source condition')
savefig_stem(fig, 'step3_best_direction_heatmap')
plt.show()

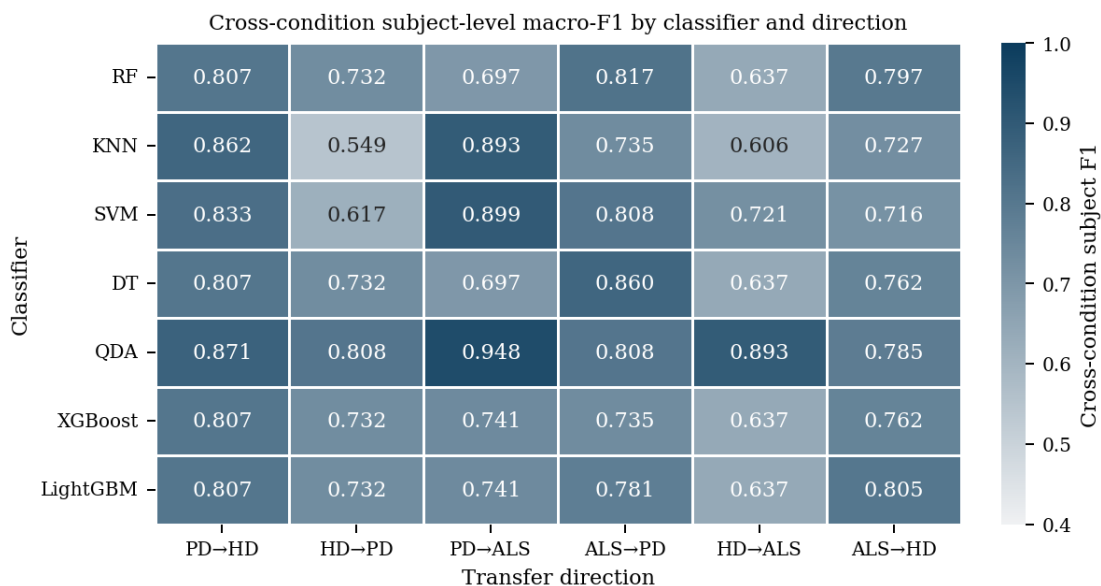
```

Saved figure: report/figures/v4/pdf/step3/degradation_heatmap.pdf

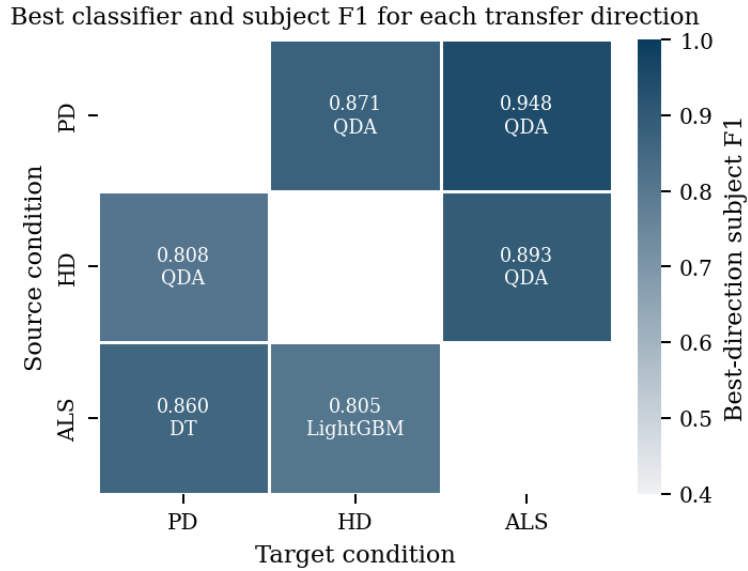
Saved figure: report/figures/v4/png/step3/degradation_heatmap.png



Saved figure: report/figures/v4/pdf/step3/cross_condition_f1_heatmap.pdf
 Saved figure: report/figures/v4/png/step3/cross_condition_f1_heatmap.png



Saved figure: report/figures/v4/pdf/step3/step3_best_direction_heatmap.pdf
 Saved figure: report/figures/v4/png/step3/step3_best_direction_heatmap.png



1.4 Bidirectional asymmetry and interval comparison

The asymmetry bars and the interval plot are complementary. The asymmetry panel shows that transfer is not symmetric within disease pairs: the subject-level gaps remain non-trivial across **PD HD**, **PD ALS**, and **HD ALS**. The interval plot then adds uncertainty context for the best classifier in each direction without inventing a direction-level primary hypothesis test. This is where the paper should say that transfer is **directionally asymmetric**, not merely degraded on average.

```
[5]: asym_rows = []
for a, b in [('pd', 'hd'), ('pd', 'als'), ('hd', 'als')]:
    ab = best_by_direction[(best_by_direction['source'] == a) &
    ↪(best_by_direction['target'] == b)].iloc[0]
    ba = best_by_direction[(best_by_direction['source'] == b) &
    ↪(best_by_direction['target'] == a)].iloc[0]
    asym_rows.append({
        'pair': f'{a.upper()} {b.upper()}',
        'subject_f1_gap': float(ab['subject_f1'] - ba['subject_f1']),
        'matched_degradation_gap': float(ab['delta_subject'] -
    ↪ba['delta_subject']),
    })
asym_df = pd.DataFrame(asym_rows)
save_table_csv(asym_df, 'step3_bidirectional_asymmetry_v4.csv')

fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.2, 2.8),
    ↪constrained_layout=True)
x = np.arange(len(asym_df))
```

```

ax.bar(x - 0.17, asym_df['subject_f1_gap'], width=0.34, label='Subject F1 gap',
      color='#4C78A8', edgecolor='black', linewidth=0.5)
ax.bar(x + 0.17, asym_df['matched_degradation_gap'], width=0.34, label='Matched_
      degradation gap', color='#9EC5E5', edgecolor='black', linewidth=0.5)
ax.axhline(0.0, color='black', linewidth=0.7)
ax.set_xticks(x)
ax.set_xticklabels(asym_df['pair'])
ax.set_ylabel('Forward minus reverse')
ax.set_title('Bidirectional asymmetry')
ax.grid(axis='y', alpha=0.2)
ax.legend(loc='center left', bbox_to_anchor=(1.02, 0.5), frameon=False)
savefig_stem(fig, 'step3_bidirectional_asymmetry')
plt.show()

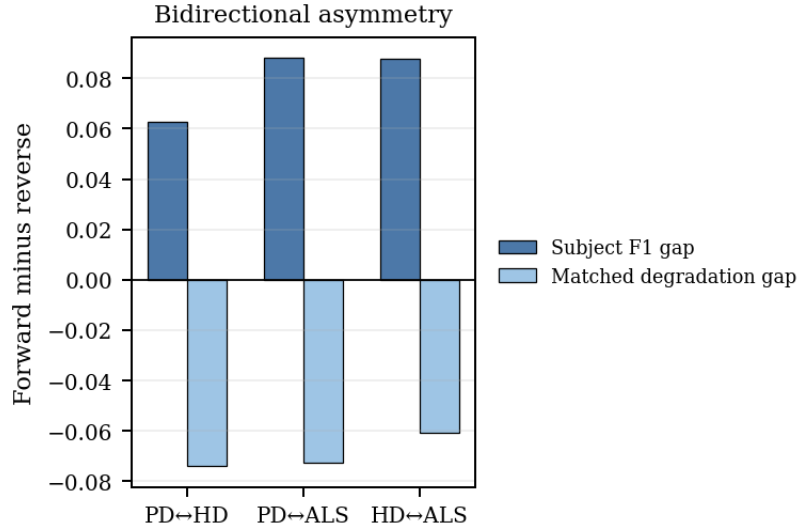
plot_df = best_by_direction.sort_values('subject_f1').reset_index(drop=True)
y = np.arange(len(plot_df))
fig, ax = plt.subplots(figsize=(IEEE_TWO_COL * 0.88, 3.2),
      constrained_layout=True)
ax.errorbar(
    plot_df['subject_f1'],
    y,
    xerr=[plot_df['subject_f1'] - plot_df['subject_ci_low'],
      plot_df['subject_ci_high'] - plot_df['subject_f1']],
    fmt='o',
    markersize=6,
    color='#1F618D',
    ecolor='#5D6D7E',
    elinewidth=1.3,
    capsize=3.2,
)
ax.set_yticks(y)
ax.set_yticklabels([f"{row.source.upper()}→{row.target.upper()} ({CLF_LABEL[row.
      classifier]})" for row in plot_df.itertuples()])
ax.set_xlabel('Subject-level macro-F1')
ax.set_title('Best-direction bootstrap intervals')
ax.grid(axis='x', alpha=0.2)
savefig_stem(fig, 'step3_direction_intervals')
plt.show()

```

Saved table: report/tables/v4/step3_bidirectional_asymmetry_v4.csv

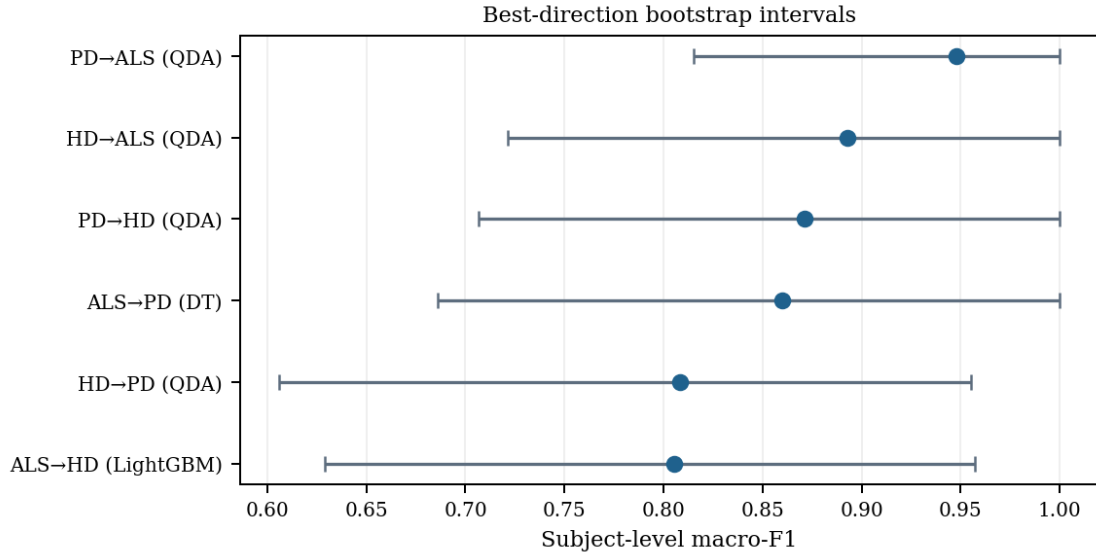
Saved figure: report/figures/v4/pdf/step3/step3_bidirectional_asymmetry.pdf

Saved figure: report/figures/v4/png/step3/step3_bidirectional_asymmetry.png



Saved figure: `report/figures/v4/pdf/step3/step3_direction_intervals.pdf`

Saved figure: `report/figures/v4/png/step3/step3_direction_intervals.png`



1.5 Within-vs-cross comparison, confusion matrices, and recall tables

These detailed diagnostics are retained for repository depth because they explain *how* a direction succeeds or fails. The within-vs-cross panel shows whether a transfer loss is a modest drop from a very strong source benchmark or a genuinely weak absolute transfer outcome. The subject-level confusion matrices and recall/specificity summaries then show which class is giving way. For example, the best **ALS**→**HD** result achieves perfect control specificity but lower HD recall, whereas

HD→ALS achieves full ALS recall with lower control specificity. Those trade-offs matter when interpreting asymmetry.

```
[6]: fig, axes = plt.subplots(2, 3, figsize=(IEEE_TWO_COL, 5.35),
    ↪gridspec_kw={'hspace': 0.50, 'wspace': 0.34})
axes = axes.ravel()
subject_color = '#1F618D'
stride_color = '#C44E52'
for ax, direction in zip(axes, DIRECTIONS):
    row = best_by_direction[best_by_direction['direction'] == direction].iloc[0]
    x_centers = np.array([0.0, 1.0])
    offset = 0.12
    ax.errorbar(
        [x_centers[0] - offset],
        [row['within_subject_f1']],
        fmt='o',
        color=subject_color,
        ecolor=subject_color,
        elinewidth=1.1,
        capsize=3.0,
        markersize=5.6,
        label='Within subject' if direction == DIRECTIONS[0] else None,
    )
    ax.errorbar(
        [x_centers[0] + offset],
        [row['subject_f1']],
        yerr=[[row['subject_f1'] - row['subject_ci_low']],
    ↪[row['subject_ci_high'] - row['subject_f1']]],
        fmt='o',
        markerfacecolor='white',
        markeredgecolor=subject_color,
        color=subject_color,
        ecolor=subject_color,
        elinewidth=1.1,
        capsize=3.0,
        markersize=5.6,
        label='Cross subject' if direction == DIRECTIONS[0] else None,
    )
    ax.errorbar(
        [x_centers[1] - offset],
        [row['within_stride_f1']],
        fmt='s',
        color=stride_color,
        ecolor=stride_color,
        elinewidth=1.0,
        capsize=3.0,
        markersize=5.1,
```

```

        label='Within stride' if direction == DIRECTIONS[0] else None,
    )
    ax.errorbar(
        [x_centers[1] + offset],
        [row['stride_f1']],
        yerr=[[row['stride_f1'] - row['stride_ci_low']], [row['stride_ci_high'] -
↪ row['stride_f1']]],
        fmt='s',
        markerfacecolor='white',
        markeredgecolor=stride_color,
        color=stride_color,
        ecolor=stride_color,
        elinewidth=1.0,
        capsize=3.0,
        markersize=5.1,
        label='Cross stride' if direction == DIRECTIONS[0] else None,
    )
    ax.set_xticks(x_centers)
    ax.set_xticklabels(['Subject', 'Stride'])
    ax.set_xlim(-0.35, 1.35)
    ax.set_ylim(0.35, 1.02)
    ax.set_title(f"{DIR_LABEL[direction]} / {CLF_LABEL[row['classifier']]}",
↪ fontsize=8.4, pad=6)
    ax.grid(axis='y', alpha=0.2)
handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, loc='lower center', bbox_to_anchor=(0.5, 0.035),
↪ ncol=4, frameon=False)
fig.suptitle('Within-versus-cross leader comparison by transfer direction', y=0.
↪ 97)
fig.subplots_adjust(top=0.89, bottom=0.15)
savefig_stem(fig, 'step3_within_vs_cross_by_direction')
plt.show()

fig, axes = plt.subplots(
    2, 3,
    figsize=(IEEE_TWO_COL, 4.85),
    gridspec_kw={'hspace': 0.22, 'wspace': 0.62}
)
axes = axes.ravel()
for idx, (ax, direction) in enumerate(zip(axes, DIRECTIONS)):
    row = best_by_direction[best_by_direction['direction'] == direction].iloc[0]
    subject_records =
↪ cross_results[direction]['classifiers'][row['classifier']]['subject_records']
    y_true = [record['y_true'] for record in subject_records]
    y_pred = [record['y_pred'] for record in subject_records]
    cm = confusion_matrix(y_true, y_pred, normalize='true')

```

```

    disp = ConfusionMatrixDisplay(cm, display_labels=['Control B',
    ↪COND_LABEL[row['target']]])
    disp.plot(ax=ax, cmap='Blues', colorbar=False, values_format='.2f')
    ax.set_title(f"{DIR_LABEL[direction]} / {CLF_LABEL[row['classifier']]}",
    ↪fontsize=8.2, pad=7)
    ax.tick_params(axis='both', labelsize=7, pad=1)
    ax.xaxis.labelpad = 2
    ax.yaxis.labelpad = 2
    if idx not in (0, 3):
        ax.set_ylabel('')
    ax.grid(False)
fig.suptitle('Subject-level normalized confusion matrices for the best
    ↪classifier in each direction', y=0.97)
fig.subplots_adjust(top=0.89, bottom=0.08)
savefig_stem(fig, 'step3_subject_confusion_matrices')
plt.show()

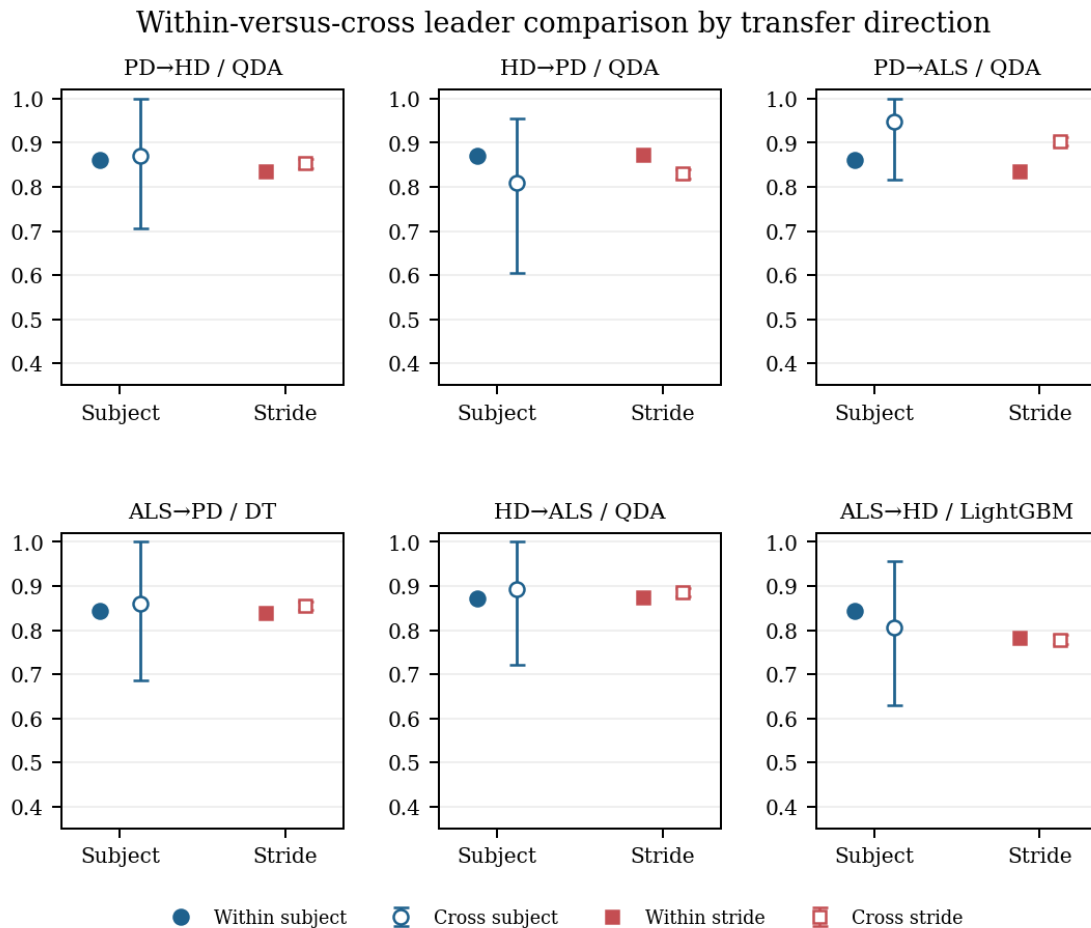
recall_df = best_by_direction[[
    'direction_label', 'classifier', 'subject_f1', 'delta_subject',
    'control_specificity_subject', 'target_recall_subject',
    'control_specificity_stride', 'target_recall_stride', 'selected_strategy'
]].copy()
recall_df['Classifier'] = recall_df['classifier'].map(CLF_LABEL)
save_table_csv(recall_df, 'step3_best_direction_recall_summary_v4.csv')
display(
    recall_df[['direction_label', 'Classifier', 'subject_f1', 'delta_subject',
    ↪'control_specificity_subject', 'target_recall_subject', 'selected_strategy']]
    .rename(columns={
        'direction_label': 'Direction',
        'subject_f1': 'Subject F1',
        'delta_subject': 'Matched degradation (within - cross)',
        'control_specificity_subject': 'Control specificity',
        'target_recall_subject': 'Disease recall',
        'selected_strategy': 'Selected strategy',
    })
    .style
    .format({
        'Subject F1': '{:.4f}',
        'Matched degradation (within - cross)': '{:+.4f}',
        'Control specificity': '{:.4f}',
        'Disease recall': '{:.4f}',
    })
    .background_gradient(subset=['Subject F1'], cmap='Blues', vmin=0.45, vmax=1.
    ↪0)
    .set_caption('Best-classifier recall and specificity summary by transfer
    ↪direction')

```


)

Saved figure: report/figures/v4/pdf/step3/step3_within_vs_cross_by_direction.pdf

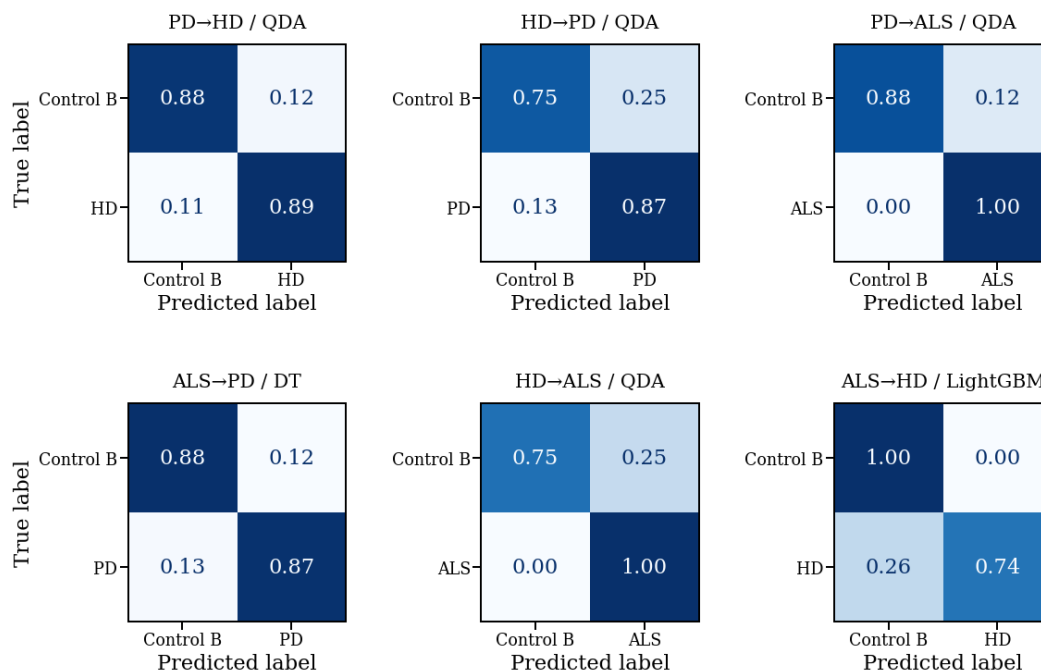
Saved figure: report/figures/v4/png/step3/step3_within_vs_cross_by_direction.png



Saved figure: report/figures/v4/pdf/step3/step3_subject_confusion_matrices.pdf

Saved figure: report/figures/v4/png/step3/step3_subject_confusion_matrices.png

Subject-level normalized confusion matrices for the best classifier in each direction



Saved table: report/tables/v4/step3_best_direction_recall_summary_v4.csv

<pandas.io.formats.style.Styler at 0x7f3782f14230>

1.6 Reporting boundaries

Subject-level matched degradation and subject-level F1 are the primary Step 3 summaries. In this notebook, `matched degradation` = `within-source subject F1` - `cross-condition transfer subject F1`, so the sign convention is fixed and should stay fixed across figures, tables, and manuscript text. The safest central claim is: **direction-level mean subject degradation is positive across all six transfer directions, while individual classifier-direction pairs still vary**. Zero-shot means the model is trained on the **source condition plus controls** and evaluated directly on the **target condition plus the disjoint control partition**, without target-condition retraining.

In manuscript prose, this can also be written explicitly as **within-condition source baseline - cross-condition transfer subject F1**; the sign convention is identical and should remain fixed throughout the project.